CS 360
Programming Languages
Day 28

Review

- Lexers / Tokenizers
- Parsers / Parse Trees / S-Expressions
- Readers / `read-syntax` / modules
- Expanders / macros / `#%module-begin`

Final Project Expectations

- **Schedule Checkpoint 1 with me now:**
  - Send push updates to your project in advance of our meeting!
  - Tell me who the group leader is!

- Expect 20 minutes:
  - 5 minute summary / demo of progress
  - 15 minutes of questions: *everyone* should be able to talk about what they worked on

- **I expect:**
  - Functional tokenizer
  - Functional parser
  - Expander stub
  - Some Racket functionality
  - 2 example programs, which your reader can parse and display

Resources

- See **final_project_resources.md** in the projects folder:
  - “Master Recipe”, including installing your language as a package
  - **Syntax Patterns**
  - **Syntax Coloring**
  - **Error Handling**
  - Variables
  - (optional) Handling Indentation
  - (optional) Adding Buttons in DrRacket
  - (optional) Unit Testing

Advice

- **Assign Responsibilities:**
  - Grammar / Parser
  - Lexer / Tokenizer
  - Reader
  - Expander: Macros and Syntax Patterns
  - Racket Functionality
    - Module X, Module Y, Module Z, etc
    - Example Programs that run purely on Racket code
  - Example Programs in your language
- **You have all the tools you need: you are adults who are responsible for your own learning and progress!!**
- I am happy to help ANY TIME! ☺

basic

- Execution is based on **LINE NUMBERS**, which are explicitly typed by the user (very unusual!)
- **goto** jumps to a specific line number (also unusual!)
- **:** separates commands on the same line (also unusual!)
- **rem** is a comment (also unusual!)
- **end** terminates the program

Example 1

```
30 rem print 'ignored'
35
50 print "never gets here"
40 end
60 print 'three' : print 1.0 + 3
70 goto 11. + 18.5 + .5 rem ignored
10 print "o" ; "n" ; "e"
20 print : goto 60.0 : end
```

Example 2

```
10 LET MAX = 5000
20 LET X = 1 : LET Y = 1
30 IF (X > MAX) GOTO 100
40 PRINT X
50 X = X + Y
60 IF (Y > MAX) GOTO 100
70 PRINT Y
80 Y = X + Y
90 GOTO 30
100 END
```

Lexing: two major changes

1. Separate lexer module:
   - What is the main reason we're separating our lexer?
2. lexer-srcloc:
   - What are **source locations**? Why are the useful?
   - What are the rules for this lexer?
   - Why can we no longer call (next-token) when we want to skip?
   - What can we do instead?
   - What are the keywords? How do we handle them?
   - Why don't we return keywords as raw lexemes any more?
   - It would be convenient to define sub-patterns for readability. How can we do that?
   - How can we match decimal numbers?
   - How do we match strings?
   - How can we test the lexer on its own?